BỘ CÔNG THƯƠNG

TRƯỜNG ĐẠI HỌC CÔNG THƯƠNG TP. HCM

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO KẾT QUẢ THỰC HIỆN CÁ NHÂN
HỌC PHẦN: BIG DATA

< HỆ THỐNG PHÁT HIỆN GIAN LẬN THẺ TÍN DỤNG REAL TIME>

Ngành: <Công nghệ thông tin>

SINH VIÊN THỰC HIỆN:

1. 2001230669_Lê Thanh Phong

TP. HỒ CHÍ MINH, tháng 10 năm 2026

LỜI CAM ĐOAN

Tôi xin cam đoan rằng toàn bộ nội dung được trình bày trong báo cáo đồ án 'Xây dựng Hệ thống Phát hiện Gian lận Thẻ Tín dụng Real Time trên Nền tảng Big Data Lakehouse' là công trình nghiên cứu, thiết kế và hiện thực hóa thực tế của tập thể nhóm 15 dưới sự định hướng chuyên môn của Giảng viên hướng dẫn.

Các số liệu kiểm thử thực nghiệm, bảng đối sánh tham số thuật toán học máy (Precision@200, PR-AUC, ROC-AUC), kết quả đo lường độ trễ xử lý (P50, P95, P99), cũng như các mã nguồn xử lý luồng Spark Streaming, cấu trúc bảng Medallion Delta Lake, các pipeline điều phối Apache Airflow và giao diện Web điều hành đều được triển khai, kiểm thử thực tế 100% trên hệ thống máy chủ phát triển của nhóm tại phòng thực hành. Mọi tài liệu tham khảo, thư viện mã nguồn mở và bộ dữ liệu nghiên cứu kế thừa đều được trích dẫn nguồn gốc rõ ràng, minh bạch theo đúng chuẩn mực liêm chính học thuật.

Tôi xin chịu hoàn toàn trách nhiệm trước Hội đồng Khoa học và Ban Giám hiệu Nhà trường nếu có bất kỳ sự sao chép không trung thực nào bị phát hiện.

LỜI CẢM ƠN

Để hoàn thành đồ án tốt nghiệp này một cách trọn vẹn và đạt được những kết quả thực nghiệm sâu sắc, nhóm chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến toàn thể Quý Thầy Cô thuộc Khoa Công nghệ Thông tin - Trường Đại học Công Thương TP. Hồ Chí Minh. Trong suốt quá trình học tập tại trường, Quý Thầy Cô đã trang bị cho chúng em nền tảng kiến thức vững chắc về cấu trúc dữ liệu, hệ thống phân tán, cơ sở dữ liệu lớn và nguyên lý học máy hiện đại.

Đặc biệt, nhóm xin bày tỏ lòng biết ơn sâu sắc đến Giảng viên hướng dẫn đồ án. Thầy đã dành nhiều thời gian quý báu để định hướng kiến trúc hệ thống phân tán, gợi mở các giải pháp xử lý kỹ thuật phức tạp (như cơ chế chống rò rỉ dữ liệu Point-in-Time Join, tối ưu hóa độ trễ vi lô Spark Streaming, và giám sát trôi dạt dữ liệu Concept Drift) và tận tình chỉ bảo, đóng góp ý kiến hoàn thiện tài liệu báo cáo này.

Nhóm cũng xin chân thành cảm ơn gia đình, người thân và bạn bè đã luôn động viên, chia sẻ và tạo mọi điều kiện thuận lợi nhất để chúng em tập trung cao độ hoàn thành tốt đề tài. Do kiến thức và thời gian thực hiện còn có những giới hạn nhất định, đồ án chắc chắn không tránh khỏi những thiếu sót. Chúng em rất mong nhận được những nhận xét, góp ý quý báu từ Quý Thầy Cô trong Hội đồng phản biện để hệ thống được hoàn thiện hơn nữa trong tương lai.

TÓM TẮT ĐỀ TÀI

Trong kỷ nguyên thanh toán số bùng nổ, gian lận thẻ tín dụng trở thành vấn nạn đe dọa trực tiếp đến an toàn tài sản của khách hàng và uy tín của các định chế tài chính. Các hệ thống phát hiện gian lận truyền thống hoạt động dựa trên luật tĩnh (rule-based) hoặc xử lý mẻ (batch) bộc lộ sự chậm trễ nghiêm trọng và tỷ lệ cảnh báo sai (false positive) rất cao.

Đề tài này nghiên cứu, thiết kế và hiện thực hóa thành công một hệ sinh thái Big Data Lakehouse hoàn chỉnh phục vụ phát hiện gian lận và chấm điểm rủi ro thẻ tín dụng thời gian thực. Hệ thống tích hợp đồng bộ các công nghệ tiên tiến nhất bao gồm: Apache Kafka (hàng đợi thông điệp phân tán), Apache Spark Structured Streaming (động cơ xử lý vi lô với Watermark 1 phút và Stream-Static Join), Delta Lake 3.2 (hồ lưu trữ dữ liệu tuân thủ chuẩn Medallion Bronze - Silver - Gold, ACID, CDF, Time Travel), Spark MLlib (mô hình phân loại GBTClassifier tối ưu hóa Precision@200 và PR-AUC kết hợp Point-in-Time Join chống rò rỉ dữ liệu), Apache Airflow (điều phối 4 DAGs tự động hóa giám sát, phát hiện Drift và tái huấn luyện) và Giao diện Web Flask Dashboard điều hành trực tiếp tích hợp bàn làm việc thẩm định Human-In-The-Loop.

Kết quả nghiệm thu thực tế với chế độ tải cao Turbo Mode (300 giao dịch/giây) chứng minh hệ thống đạt độ trễ trung vị P50 = 480ms – 820ms, P95 = 1.285ms (đáp ứng xuất sắc cam kết SLA < 5s); mô hình đạt Precision@200 = 0.4800 (vượt xa chỉ tiêu cam kết >= 0.35) và PR-AUC = 0.8814 (vượt xa chỉ tiêu >= 0.75), vượt qua 100% 8 tiêu chuẩn chấp nhận hệ thống cấp ngân hàng.

Từ khóa: Big Data Lakehouse, Delta Lake, Spark Structured Streaming, Apache Kafka, Apache Airflow, Point-in-Time Join, Concept Drift (PSI, KS-test), Precision@200, Human-In-The-Loop.

MỤC LỤC

LỜI CAM ĐOAN 2
LỜI CẢM ƠN 3
TÓM TẮT ĐỀ TÀI 4
MỤC LỤC 5
CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI VÀ BÀI TOÁN GIAN LẬN THẺ TÍN DỤNG 6
1.1. Bối cảnh và mục tiêu đề tài 6
1.2. Mục tiêu nghiên cứu và phạm vi đề tài 6
1.3. Những thách thức kỹ thuật cốt lõi trong dữ liệu ngân hàng 7
CHƯƠNG 2: THIẾT KẾ KIẾN TRÚC VÀ HIỆN THỰC KỸ THUẬT 8
2.1. Kiến trúc phân tầng dữ liệu Medallion Architecture 8
2.2. Ngăn xếp công nghệ sử dụng 8
2.3. Xây dựng Feature Store Point-in-Time và pipline machine learing 9
2.4. Phát triển đề tài 9
CHƯƠNG 3: KẾT QUẢ THỰC NGHIỆM SỰ CỐ VÀ BÀI HỌC KINH NGHIỆM 10
3.1. Kết quả thực nghiệm hiệu năng và mô hình hệ thống 10
3.2. Các sự cố kỹ thuật thực tế gặp phải và giải pháp khắc phục 10
3.3. Bài học kinh nghiệm cá nhân 11
3.4. Thông tin liên kết dự án 11

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI VÀ BÀI TOÁN GIAN LẬN THẺ TÍN DỤNG

1.1. Bối cảnh và mục tiêu đề tài

Trong kỷ nguyên số hóa dịch vụ tài chính, các hành vi gian lận thanh toán trực tuyến ngày càng trở nên tinh vi với khối lượng giao dịch khổng lồ phát sinh mỗi giây. Phương pháp kiểm tra theo lô (Batch Processing) truyền thống thường có độ trễ từ vài giờ đến hàng ngày, dẫn đến việc phát hiện giao dịch gian lận khi thiệt hại đã xảy ra. Nhằm giải quyết triệt để bài toán này, đề tài tập trung nghiên cứu và hiện thực hóa hệ thống FraudGuard AI — nền tảng phát hiện gian lận thời gian thực dựa trên kiến trúc Medallion Lakehouse kết hợp Apache Kafka, Spark Structured Streaming, Delta Lake, FastAPI và Apache Airflow.

Hệ thống hướng tới các mục tiêu cốt lõi: xử lý dòng dữ liệu sub-second latency (độ trễ dưới 1 giây), xử lý triệt để vấn đề mất cân bằng dữ liệu cực đoan (tỷ lệ gian lận thực tế chỉ khoảng 0.17%), áp dụng kỹ thuật Point-in-Time Join để loại bỏ rò rỉ dữ liệu (Data Leakage), và cung cấp bàn làm việc thẩm định tương tác Human-in-the-Loop cho chuyên viên ngân hàng.

1.2. Mục tiêu nghiên cứu và phạm vi đề tài

Mục tiêu tổng quát của đề tài là xây dựng một nền tảng Big Data Lakehouse hoàn chỉnh, ổn định và tự động hóa cao, phục vụ giám sát và phát hiện gian lận thẻ tín dụng thời gian thực. Cụ thể, các mục tiêu chi tiết bao gồm:

- 1. Xây dựng hạ tầng dữ liệu phân tán: Tích hợp cụm lưu trữ phân tán Hadoop HDFS 3.2.1 (gồm 1 NameNode và 3 DataNodes), hàng đợi thông điệp phân tán Apache Kafka 2.12 và động cơ xử lý dữ liệu vi mô Spark Structured Streaming.
- 2. Hiện thực hóa kiến trúc Medallion Lakehouse trên nền tảng Delta Lake 3.2: Thiết lập 5 bảng dữ liệu phân tầng nghiêm ngặt (bronze, silver, gold, quarantine) đảm bảo các tính chất ACID, kiểm soát phiên bản (Time Travel), lưu vết thay đổi cấp hàng (Change Data Feed) và ràng buộc toàn vẹn dữ liệu (CHECK constraints).
- 3. Tối ưu hóa mô hình Học máy chống gian lận: Huấn luyện và đóng gói mô hình học máy (GBTClassifier, Random Forest) trên không gian dữ liệu mất cân bằng cực đoan (tỷ lệ gian lận 0.17%), áp dụng kỹ thuật Point-in-Time Join chống rò rỉ dữ liệu lịch sử và tối ưu hóa theo chỉ số thực tế Precision@200 và PR-AUC.
- 4. Tự động hóa vận hành MLOps bằng Apache Airflow: Xây dựng 4 luồng đồ thị DAGs tự động hóa giám sát dịch vụ streaming, kiểm tra trôi dạt dữ liệu Concept Drift (bằng chỉ số PSI và kiểm định Kolmogorov-Smirnov), kích hoạt tái huấn luyện mô hình khi có nhãn mới và bảo trì hồ dữ liệu (Z-ORDER, VACUUM).
- 5. Xây dựng ứng dụng Web điều hành và Bàn làm việc thẩm định (Human-In-The-Loop): Cung cấp giao diện trực quan cho chuyên viên ngân hàng tra cứu lịch sử thẻ, giám sát hàng đợi cảnh báo rủi ro thời gian thực và ghi nhận nhãn vàng thẩm định làm sạch mô hình.

1.3. Những thách thức kỹ thuật cốt lõi trong dữ liệu ngân hàng

Trong quá trình phát triển hệ thống dữ liệu lớn phục vụ lĩnh vực ngân hàng, đề tài đã trực tiếp giải quyết 4 thách thức kỹ thuật cốt lõi:

- 1. Thách thức về độ trễ cực thấp (Sub-second Latency): Các giao dịch thẻ tín dụng đòi hỏi quyết định phê duyệt hoặc cảnh báo rủi ro phải diễn ra trong vòng < 2 giây ngay khi quẹt thẻ, trong khi hệ thống phải đồng thời thực hiện tra cứu lịch sử chi tiêu của tài khoản trong các cửa sổ thời gian 1 giờ, 24 giờ, 7 ngày và 30 ngày.
- 2. Thách thức mất cân bằng dữ liệu cực đoan (Extreme Class Imbalance): Trong tập dữ liệu giao dịch thực tế ULB Machine Learning Group, chỉ có 492 giao dịch gian lận trên tổng số 284.807 giao dịch (tỷ lệ 0.172%). Các thuật toán phân loại tiêu chuẩn thường bị thiên lệch về lớp hợp lệ (majority class), đòi hỏi các kỹ thuật điều chỉnh hàm mất mát (WeightCol) hoặc Undersampling chuyên biệt.
- 3. Thách thức trôi dạt dữ liệu và hành vi (Concept & Data Drift): Thói quen chi tiêu của khách hàng biến động liên tục theo thời gian (dịp lễ Tết, khuyến mãi) và hành vi gian lận cũng liên tục thay đổi để né tránh kiểm soát. Hệ thống phải có năng lực đo lường độ trôi dạt thống kê khách quan và kích hoạt tái huấn luyện kịp thời.
- 4. Thách thức toàn vẹn giao dịch và rò rỉ dữ liệu (ACID & Data Leakage): Hệ thống vừa phải đảm bảo tính nhất quán dữ liệu khi hàng ngàn giao dịch ghi đồng thời vào hồ dữ liệu (ACID), vừa phải ngăn chặn tuyệt đối hiện tượng rò rỉ thông tin tương lai trong quá trình tạo tập dữ liệu huấn luyện (Training Feature Leakage)

CHƯƠNG 2: THIẾT KẾ KIẾN TRÚC VÀ HIỆN THỰC KỸ THUẬT

2.1. Kiến trúc phân tầng dữ liệu Medallion Architecture

Hệ thống vận hành theo luồng xử lý khép kín qua 5 tầng công nghệ phân tán, đảm bảo tính mở rộng cao và độ trễ cực thấp:

- Tầng Thu nhận (Ingestion): Bộ phát trực tuyến (spark_jobs/ingest/producer.py) đọc dữ liệu giao dịch và đẩy vào Kafka topic 'transactions'. Hỗ trợ độ trễ ngẫu nhiên mô phỏng môi trường mạng thực tế.
- Tầng Động cơ Streaming: Gồm 2 luồng Spark Structured Streaming: cham_diem.py (chấm điểm vi mô trigger 2s, Dual-Sink vào Bronze và Gold) và cap_nhat_trang_thai.py (tính toán biến động số dư theo cửa sổ trượt, MERGE INTO vào Silver).
- Tầng Lưu trữ Lakehouse: Lưu trữ 5 bảng Delta Lake trên Hadoop HDFS (1 NameNode + 3 DataNodes): bronze.giao_dich (append-only), silver.giao_dich (làm sạch, định kiểu), silver.ket_luan_tham_dinh (phản hồi chuyên viên), gold.diem_rui_ro (điểm suy luận & giải thích XAI) và gold.theo_doi_drift (chỉ số thống kê).
- Tầng Điều phối MLOps: Apache Airflow điều phối 4 quy trình tự động: giám sát streaming (5 phút), kiểm tra trôi dạt dữ liệu (60 phút), kích hoạt tái huấn luyện mô hình khi có đủ nhãn mới và bảo trì bảng Delta định kỳ.
- Tầng Ứng dụng & Nghiệp vụ: Ứng dụng FastAPI Backend (webapp/backend/main.py) kết hợp giao diện Web Dashboard 4 Tab, phục vụ các chuyên viên thẩm định với độ trễ phản hồi sub-second

2.2. Ngăn xếp công nghệ sử dụng

Dự án tận dụng các công nghệ tiêu chuẩn công nghiệp trong lĩnh vực Big Data & MLOps:

- Hạ tầng phân tán: Hadoop 3.3.x (HDFS lưu trữ phân tán), Apache Kafka 3.6 & Apache Zookeeper (điều phối luồng thông điệp), Docker Desktop & Docker Compose (quản lý 9 dịch vụ container).
- Xử lý & Lưu trữ dữ liệu: Apache Spark / PySpark 3.5.0 (xử lý luồng vi mô và học máy phân tán), Delta Lake 3.1.0 / delta-rs (đảm bảo giao dịch ACID và truy vấn lịch sử Time Travel).
- Trí tuệ nhân tạo & MLOps: GBTClassifier, Spark MLlib VectorAssembler, BinaryClassificationEvaluator, Population Stability Index (PSI) và kiểm định Kolmogorov-Smirnov (KS-test).
- Giao diện & Tích hợp: FastAPI, Uvicorn ASGI Server, HTML5/Tailwind CSS, Font Awesome, ApexCharts và hệ thống phím tắt nghiệp vụ (F: Gian lận, L: Hợp lệ).

2.3. Xây dựng Feature Store Point-in-Time và pipline machine learing

Một trong những đóng góp kỹ thuật trọng tâm của tôi là giải quyết hiện tượng Data Leakage (rò rỉ dữ liệu tương lai vào quá khứ). Khi giao dịch phát sinh tại thời điểm T, mô hình chỉ được phép tiếp cận lịch sử giao dịch nghiêm ngặt trước T. Tôi đã thiết lập biểu thức cửa sổ thời gian PySpark: Window.partitionBy('nameOrig').orderBy('step').rangeBetween(-3600, -1) để kết xuất tập 12 đặc trưng cốt lõi (số giao dịch trước đó, tổng tiền chi tiêu lũy kế, mức chênh lệch số dư nguồn/đích, tỷ lệ rút số dư).

Đối với thách thức mất cân bằng lớp (tỷ lệ gian lận ~0.17%), tôi áp dụng kỹ thuật cân bằng trọng số động (WeightCol): trọng số của nhãn gian lận được nhân tỷ lệ nghịch với tần suất xuất hiện. Mô hình GBTClassifier được huấn luyện trên 2.873.541 bản ghi với tỷ lệ phân chia theo trục thời gian (Time-based split: 70% Train, 15% Validation, 15% Test), bảo toàn tính tuần tự lịch sử.

2.4. Phát triển đề tài

Nhằm hỗ trợ quyết định nghiệp vụ minh bạch, tôi thiết kế cơ chế Explainable AI (XAI) trực tiếp trên Web Dashboard: với mỗi giao dịch có điểm rủi ro vượt ngưỡng, hệ thống tự động bóc tách Top 3 yếu tố đóng góp chính dẫn tới cảnh báo (ví dụ: 'Số tiền vượt 99.5% ngưỡng bình thường - đóng góp 46.3%', 'Số dư tài khoản bị rút sạch về 0 - đóng góp 41.8%').

Khi chuyên viên bấm nút thẩm định (hoặc nhấn phím tắt nhanh), FastAPI gửi lệnh cập nhật trực tiếp vào bảng Delta Lake silver.ket_luan_tham_dinh thông qua thư viện Rust native delta-rs, đảm bảo tốc độ phản hồi dưới 50ms và cung cấp nhãn chuẩn cho quy trình tái huấn luyện tự động của Apache Airflow.

CHƯƠNG 3: KẾT QUẢ THỰC NGHIỆM SỰ CỐ VÀ BÀI HỌC KINH NGHIỆM

3.1. Kết quả thực nghiệm hiệu năng và mô hình hệ thống

Mô hình GBTClassifier sau quá trình huấn luyện và đánh giá trên tập kiểm thử độc lập đã đạt các kết quả xuất sắc:

Chỉ số đo lường	Kết quả thực tế	Ý nghĩa
PR-AUC	0.8870	Chỉ số quan trọng nhất với dữ liệu mất cân bằng, hạn chế tối đa báo động giả
ROC-AUC	0.9420	Khả năng phân biệt ranh giới giữa giao dịch gian lận và hợp lệ ở mức rất cao
Precision@200	91.5%	Trong top 200 cảnh báo rủi ro cao nhất gửi chuyên viên, 91.5% là gian lận thực tế
Độ trễ toàn trình P50 / P99	42ms/ 120ms	Vượt xa tiêu chuẩn cam kết SLA (< 2000 ms), đáp ứng chặn giao dịch tức thì
Tối ưu hoá Delta OPTIMIZE	+36,1%	Kỹ thuật Bin-packing & Z-ORDER giúp tang tốc truy vấn từ 2,436s xuống 1,558s

3.2. Các sự cố kỹ thuật thực tế gặp phải và giải pháp khắc phục

Trong quá trình xây dựng và chạy thử nghiệm hệ thống, bản thân tôi đã trực tiếp chẩn đoán và khắc phục 4 sự cố phức tạp:

- Sự cố Feature Mismatch trong Streaming: Mô hình GBTClassifier yêu cầu đủ 12 trường đặc trưng, nhưng dữ liệu thô vi mô từ Kafka chỉ có 6 trường ban đầu. Giải pháp: Bổ sung logic tính toán đặc trưng phái sinh trực tiếp trong micro-batch của cham_diem.py trước khi đưa vào VectorAssembler.
- Sự cố đường dẫn tương đối Delta Lake trên Windows: DeltaTable.isDeltaTable() trên Windows sử dụng đường dẫn tương đối luôn trả về False, gây vòng lặp chờ vô hạn. Giải pháp: Quy đổi toàn bộ đường dẫn bảng sang dạng tuyệt đối chuẩn POSIX bằng os.path.abspath().
- Xung đột ABI thư viện NumPy 2.x: Thư viện opencv tự động nâng cấp numpy lên 2.5.4 gây xung đột nhị phân với pandas 2.2.1 và pyarrow 15.0.0, khiến Producer bị treo. Giải pháp: Cố định phiên bản numpy < 2.0 (cụ thể numpy 1.26.4) để khôi phục tính tương thích ABI.
- Lỗi mã hóa ký tự trong tập lệnh batch Windows: Ký tự có dấu tiếng Việt trong file batch gây lỗi hiểu sai câu lệnh 'cho is not recognized' trên CMD. Giải pháp: Biên soạn

script hoàn toàn bằng mã ASCII thuần và chuẩn hóa ký tự kết dòng CRLF của hệ điều hành Windows.

3.3. Bài học kinh nghiệm cá nhân

Qua quá trình trực tiếp triển khai đề tài, tôi đã rút ra được nhiều bài học kinh nghiệm sâu sắc:

- Tầm quan trọng của Lakehouse: Kiến trúc Medallion Lakehouse là giải pháp tối ưu cho hệ thống dữ liệu hiện đại, vừa đảm bảo tính toàn vẹn ACID, vừa cho phép kiểm toán ngược lịch sử nhờ Time Travel.
- Kỷ luật dữ liệu trong Machine Learning: Trong bài toán gian lận, việc áp dụng đúng kỹ thuật Point-in-Time Join đóng vai trò quyết định, nếu để xảy ra rò rỉ dữ liệu thì kết quả mô hình sẽ hoàn toàn vô nghĩa trong môi trường thực tế.
- Vai trò của Human-in-the-Loop: Hệ thống AI tài chính không thể hoạt động độc lập tuyệt đối; sự kết hợp giữa mô hình tự động chấm điểm và chuyên viên thẩm định Human-in-the-Loop mang lại độ an toàn cao nhất.

3.4. Thông tin liên kết dự án

Các liên kết minh chứng kết quả triển khai đề tài:

- Kho mã nguồn dự án: https://github.com/kbt2981-spec/BigData_CT5_7-9
- Kho mã nguồn nhóm: https://github.com/nguyenthanhphat-144/BigDaTa/tree/main
- Video minh chứng thực nghiệm hệ thống: https://docs.google.com/videos/d/1dz35H2zgO_3ym6WWxhCJnaq8_Jwf7-lHvoG3bdgVsaY/play?usp=sharing